Sortir d’un ETL visuel : reconstruction d’un pipeline décisionnel dans Power Query et validation de son équivalence

ZIDOUR Idir — Master 2 DS4SC, Université de Tours

Tuteur universitaire : DOUMARD Emmanuel — Entreprise d’accueil : Adiwatt

Rapport de phase de réalisation — S10 — brouillon du 15/09/2026

Résumé

Ce rapport rend compte de la phase de réalisation d’un projet engagé par une étude bibliographique : supprimer l’étage de préparation de données d’une chaîne décisionnelle industrielle, en l’occurrence un workflow KNIME de 193 nœuds exécuté quatre fois par jour, qui produit douze tables dans une base intermédiaire. Nous reconstruisons son traitement dans Power Query, le moteur de transformation de Power BI. Notre méthode enchaîne une rétro-ingénierie du workflow à partir de son format de sérialisation, la construction d’une représentation intermédiaire indépendante des deux outils, la traduction de celle-ci en requêtes M, puis une validation par test différentiel contre un oracle figé, c’est-à-dire les sorties de la chaîne d’origine capturées avant toute modification. Sur douze tables, neuf sortent équivalentes ligne à ligne. Les trois autres divergent pour des raisons distinctes, que nous identifions et prouvons sur les données : une agrégation dépendante de l’ordre de lecture dans la chaîne d’origine, un départage non spécifié entre deux sources dans le modèle migré, et un nœud dont la configuration affichée ne correspond pas à son comportement réel. Le test différentiel nous a ainsi permis de découvrir deux défauts de la chaîne qu’il devait reproduire. La solution migrée est en service ; la durée du traitement passe de 4 min 33 s à 1 min 20 s, et l’unité de défaillance passe de l’exécution entière à la cellule.

1. Introduction

Un système décisionnel met en relation les bases de données des applications de gestion et les tableaux de bord que consultent les décideurs, et il faut bien que quelque chose, entre les deux, nettoie, recoupe et agrège. Dans une organisation de taille moyenne, ce travail est souvent confié à un outil de préparation à flux visuel, installé sur un poste, maintenu par une ou deux personnes, dont l’usage s’étend au fil des besoins. Le cas que nous traitons en est un exemple représentatif. Une base brute reçoit les extractions du progiciel de gestion d’Adiwatt ; un workflow KNIME de 193 nœuds et 219 connexions les lit intégralement, les nettoie, les recoupe et écrit douze tables dans une seconde base dite propre, dont un rapport Power BI de vingt-huit tables et d’une centaine de mesures est l’unique lecteur. Une tâche planifiée du système d’exploitation enchaîne l’ensemble quatre fois par jour sur une machine dédiée.

L’étude bibliographique qui a précédé ce travail a posé la question de la suppression de cet étage : une chaîne de préparation bâtie dans un outil de flux visuel peut-elle être reconstruite à l’intérieur du moteur de transformation de l’outil de restitution, sans perte d’équivalence, et à quel coût ? Nous y avions établi que les propriétés en jeu, à savoir le lieu et le moment du calcul, les garanties du modèle d’exécution, la correspondance entre pipelines et la preuve d’équivalence, sont des objets d’étude constitués, mais qu’aucun travail ne traite la traduction d’un workflow visuel vers un langage de requêtes ni l’équivalence d’une chaîne complète. Nous avions reformulé la question en trois sous-questions évaluables : quelle représentation intermédiaire de la logique métier, indépendante des deux outils, suffit à engendrer le pipeline cible ; à quel niveau d’équivalence la migration doit-elle être validée, et par quel protocole ; que gagne-t-on et que perd-on, mesurément, à supprimer l’étage intermédiaire.

Le présent rapport répond à ces trois questions par une réalisation complète, en service au moment de la rédaction. Il ne s’agit pas d’une preuve de concept sur un sous-ensemble : les douze tables ont été reconstruites, le rapport migré est actualisé en parallèle de l’ancien, et la bascule est décidée. Nous annonçons d’emblée nos trois résultats. Le premier est méthodologique : le format de sérialisation du workflow, une fois analysé, fournit une représentation intermédiaire suffisante pour engendrer la cible, et le test différentiel contre un oracle figé s’est révélé praticable à l’échelle d’une chaîne complète, au

prix d’une contrainte de calendrier que notre première tentative a matérialisée. Le deuxième est empirique : neuf tables sur douze sont équivalentes ligne à ligne, et nous expliquons les trois divergences résiduelles jusqu’au nœud responsable. Le troisième nous a surpris : deux de ces trois divergences sont des défauts de la chaîne d’origine, que le test différentiel a révélés alors qu’il cherchait à la reproduire. L’oracle nous servait de référence pour l’équivalence, mais il ne garantissait pas sa propre correction.

Le document est organisé comme suit. La section 2 rappelle le vocabulaire et le terrain. La section 3 résume l’état de l’art selon les quatre axes de l’étude bibliographique, en ne conservant que ce qui nous a effectivement servi. La section 4 décrit la rétro-ingénierie du workflow et la représentation intermédiaire que nous en tirons. La section 5 présente la reconstruction dans Power Query : architecture, règles de traduction, frontière de délégation, cas difficiles. La section 6 expose le protocole de validation et ses résultats. La section 7 dresse le bilan mesuré de la migration. La section 8 conclut et ouvre les perspectives.

2. Préliminaires

Nous fixons ici le vocabulaire employé dans la suite et décrivons le terrain. Le lecteur de notre étude bibliographique y retrouvera des définitions déjà posées, ramenées à l’essentiel.

2.1. Chaîne décisionnelle et terrain

Un système décisionnel relie des bases opérationnelles à des indicateurs consultés par des utilisateurs métier, à travers trois étages : la préparation des données, leur modélisation en un schéma interrogeable (le plus souvent en étoile) et leur restitution. La préparation est traditionnellement décrite par l’acronyme ETL : un outil tiers extrait, transforme, puis charge dans une base intermédiaire. L’approche concurrente, dite ELT, transforme les données là où elles reposent. La différence porte sur le lieu où le calcul s’exécute.

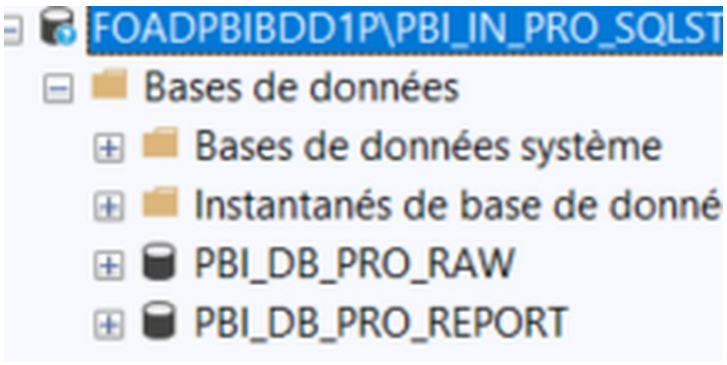


Figure 1 — Les deux bases du dispositif d’origine sur la même instance SQL Server. La base brute est alimentée par le progiciel de gestion ; la base propre est produite par KNIME et consommée par le rapport. C’est cette seconde base que la migration supprime.

L’architecture d’origine relève du premier cas (figure 1). Deux bases coexistent sur la même instance SQL Server : PBI_DB_PRO_RAW reçoit trente-six tables extraites du progiciel de gestion par des scripts d’alimentation, et PBI_DB_PRO_REPORT ne contient que les douze tables produites par le workflow à destination du rapport. Elle n’a pas d’autre usage. Une tâche planifiée Windows exécute le workflow puis déclenche l’actualisation du rapport dans le service Power BI, quatre fois par jour, sur un serveur dédié (figure 2). Elle journalise et notifie les échecs par courriel, mais ne sait pas reprendre : un traitement interrompu doit être relancé depuis son début, et le rapport reste sur les données de la veille jusqu’à ce qu’une exécution aboutisse.

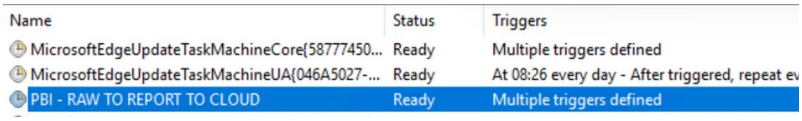


Figure 2 — La tâche planifiée « PBI - RAW TO REPORT TO CLOUD », déclenchée quatre fois par jour : exécution du workflow, puis actualisation du rapport dans le service.

Nous appelons pipeline la succession ordonnée de transformations menant d'une table source à une table exploitable, et opérateur chacune de ces transformations. Un résultat intermédiaire est matérialisé lorsqu'il est effectivement calculé et stocké, par opposition à un résultat qui n'existe que comme description d'un calcul à effectuer.

2.2. Deux modèles d'exécution

KNIME Analytics Platform exécute un graphe orienté acyclique de nœuds. Chaque nœud est un opérateur paramétré par une interface graphique, chaque arête transporte une table dont le schéma est connu, et l'exécution progresse nœud par nœud, la sortie de chacun étant matérialisée avant que le suivant ne commence. L'outil est libre et gratuit : l'enjeu de notre migration n'est pas le coût d'une licence.

Power Query est le moteur de transformation intégré à Power BI. Une requête y est une suite d'expressions nommées dans le langage M, fonctionnel, à valeurs immuables et à évaluation paresseuse. Le moteur construit d'abord une description complète du calcul, puis décide comment l'exécuter ; lorsque la source sait exécuter une opération, il la lui délègue en la traduisant en SQL. Cette délégation, que l'éditeur appelle query folding, est partielle par nature : dès qu'une étape emploie une construction que la source ne sait pas exprimer, elle et toutes les suivantes sont évaluées localement. Le point de rupture détermine le volume rapatrié.

Une troisième différence ne se révèle qu'en cas d'anomalie. Dans un flux de nœuds, l'erreur est un événement de contrôle : le nœud qui échoue arrête l'exécution, et l'unité de défaillance est l'exécution entière. Dans un langage à valeurs immuables, l'erreur est une valeur : la conversion qui échoue produit une cellule en erreur, la table continue d'exister et le traitement se poursuit. Dans la chaîne d'origine, l'échec était bien détecté et notifié, mais aucune reprise n'était possible ; dans la chaîne migrée, il n'y a plus d'exécution à reprendre.

2.3. Équivalence, tolérance et oracle

Deux pipelines sont dits équivalents si, pour les mêmes données d'entrée, ils produisent la même sortie. Nous entendons l'égalité de deux tables au sens des multi-ensembles, c'est-à-dire que les doublons comptent, et nous ignorons l'ordre des lignes puisque aucun des deux moteurs ne le garantit. L'arithmétique à virgule flottante n'étant pas associative, nous remplaçons l'égalité stricte des nombres par une tolérance déclarée avant l'expérience.

Le test logiciel appelle oracle ce qui permet de trancher si le résultat observé d'une exécution est correct. Une migration se trouve dans une situation privilégiée : la chaîne à remplacer fonctionne encore, et ses sorties fournissent la référence exacte à laquelle confronter celles du pipeline reconstruit ; comparer ainsi deux implémentations d'un même traitement est un test différentiel. Mais l'oracle est un système vivant, dont les sorties changent à chaque exécution : il faut le figer, c'est-à-dire archiver un export daté, avant toute modification. Nous verrons en section 6 que l'oracle n'est une référence que pour l'équivalence, et qu'il ne garantit pas sa propre correction.

3. État de l'art

Notre étude bibliographique a organisé la littérature selon quatre propriétés transverses plutôt que par domaine. Nous la résumons ici en ne retenant, pour chaque axe, que ce qui a effectivement outillé la réalisation, et en signalant à chaque fois le point où elle s'arrête.

3.1. Où et quand le calcul s'exécute

L'idée de confier une partie du travail à la source est formalisée par Haas et al. (1997) : dans un système interrogeant des sources hétérogènes, l'optimiseur ne doit engendrer que les plans que les capacités déclarées de chaque source savent exécuter. La délégation est une contrainte d'optimisation,

non une astuce. Simitsis et al. (2005) déplacent la question à l'intérieur du flux ETL et démontrent que certaines réécritures, comme remonter un filtre avant une jointure, préservent la sémantique. Yan et al. (2023) étendent la délégation aux opérateurs non relationnels en vérifiant formellement que la sortie demeure inchangée, et montrent que l'exercice cesse d'être rentable dès que l'opérateur n'est que partiellement délégable.

Johnston et al. (2004) décrivent les langages par flux de données comme des graphes où chaque nœud produit un résultat complet et matérialisé avant que le suivant ne commence : aucune vue d'ensemble n'existe avant l'exécution, et rien ne peut être réorganisé. Armbrust et al. (2015) donnent l'illustration inverse, avec une interface à évaluation différée doublée d'un optimiseur qui repousse filtres et projections dans la source. La paresse n'est pas une commodité d'écriture, c'est la condition de possibilité de la délégation. Cet axe nous a apporté le vocabulaire pour analyser, requête par requête, où la chaîne de délégation se rompt (section 5.3).

3.2. Ce que le modèle d'exécution garantit

Shi et al. (2016) nomment le code qui suppose déterministe une spécification qui ne l'est pas, et proposent de le détecter en explorant les ordres d'itération licites. La transposition aux données est immédiate : un opérateur qui retient la première ou la dernière valeur d'un groupe n'a de sens que si l'ordre de lecture est garanti, or ni un flux de nœuds ni un langage de transformation ne le garantissent, et aucun des deux ne le documente. Pritchard et Wicenec (2024) fournissent le vocabulaire pour dire de quelle stabilité on parle, en distinguant relance, répétition, recalcul, reproduction et réplication, et obligent à choisir le niveau d'équivalence visé avant de prétendre le vérifier. Cet axe nous a apporté la grille de lecture des deux divergences les plus instructives de la section 6, qui sont précisément des cas de non-déterminisme supposé déterministe.

3.3. Établir la correspondance entre deux pipelines

Trois stratégies coexistent. La taxonomie d'opérateurs de Vassiliadis et al. (2009), qui classe les activités ETL selon leur cardinalité en schéma et en tuples, permet de rapprocher méthodiquement un nœud du pipeline source, une étape du pipeline cible et un opérateur relationnel. Le lignage au sens de Cui et Widom (2001) fournit un critère de complétude : toute colonne produite doit être rattachable aux colonnes sources dont elle dérive, y compris lorsque la transformation est opaque. Le patron de modernisation dirigée par les modèles de Böhm et al. (2026), structuré en analyse, enrichissement, synthèse et transition autour d'un modèle pivot indépendant des technologies, fournit la démarche : ne pas traduire directement, mais reconstruire d'abord une représentation intermédiaire de la logique métier, puis engendrer la cible. Cet axe nous a apporté la structure de la section 4 et la table de correspondance de la section 5.2. En revanche, il ne fournissait aucune règle de traduction pour le couple de langages considéré, et nous avons dû les écrire.

3.4. Prouver que la migration n'a rien changé

Les approches de validation s'ordonnent selon la force de leur garantie. La preuve formelle d'équivalence (Zhou et al., 2022 ; He et al., 2024) raisonne sur des paires de requêtes, sous sémantique des multi-ensembles et avec production de contre-exemples, mais jamais sur une chaîne complète. Le test différentiel fondé sur un oracle construit (Rigger et Su, 2020) contourne l'absence de vérité de terrain en engendrant des requêtes dont une partie du résultat est connue par construction ; dans notre cas, la chaîne d'origine fournit l'oracle elle-même, à condition d'en archiver les sorties avant son démantèlement. L'assertion déclarative (Schelter et al., 2018) exprime des contraintes de qualité compilées en requêtes d'agrégation, applicables sans référence et à chaque exécution. Cet axe nous a apporté le protocole de la section 6 tout entier, transposé de la requête à la chaîne. Ce qu'il ne disait pas, et que nous établissons en section 6, c'est que l'oracle d'une migration peut être faux, et que le test différentiel est précisément l'instrument qui le montre.

Sculley et al. (2015) nous servent enfin de toile de fond : les formes de dette que contracte tout système assemblé par accumulation (code de liaison, enchevêtrement, configuration) se paient en maintenance et non en développement. La dépendance à un outil unique en est un cas particulier, qui

tient à l’absence de représentation portable de la logique métier. Notre section 4 est la construction de cette représentation.

4. Le pipeline d’origine : rétro-ingénierie et représentation intermédiaire

Notre première sous-question portait sur la représentation intermédiaire : indépendante des deux outils, elle doit être suffisante pour engendrer la cible. Nous décrivons ici comment nous l’avons obtenue et ce qu’elle nous a révélé du pipeline d’origine.

4.1. Le format de sérialisation comme seule spécification

Le workflow n’a d’autre spécification que lui-même. Nous en sommes le mainteneur, et cette proximité n’a pas supprimé la difficulté : passé quelques dizaines de nœuds, un graphe cesse d’être tenu en mémoire par qui que ce soit, et sa représentation à l’écran (figure 3) exige un facteur de zoom que la page ne restitue pas. Nous sommes donc passés par le format de sérialisation. Un workflow KNIME est un répertoire : un fichier workflow.knime décrit les nœuds et leurs connexions en XML, et chaque nœud possède son propre répertoire contenant un fichier settings.xml où sont sérialisés tous ses paramètres, c’est-à-dire les colonnes sélectionnées, les expressions, les clés de jointure, les méthodes d’agrégation et les politiques d’erreur.

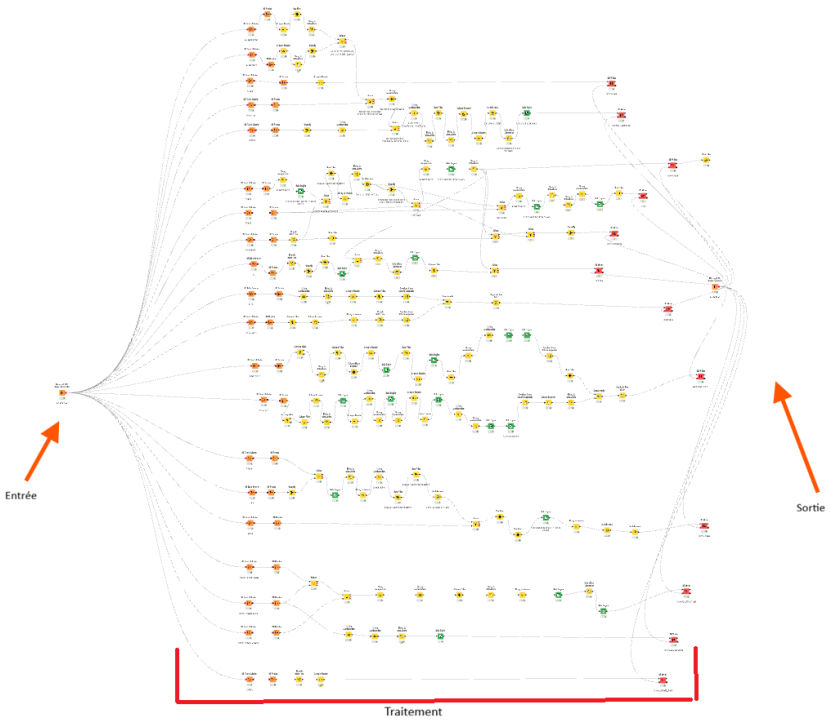


Figure 3 — Le workflow à l’ouverture de KNIME : 193 nœuds et 219 connexions. Les deux connecteurs d’entrée sont à gauche, les douze écritures à droite. Les annotations Entrée, Traitement et Sortie ont été ajoutées pour cette figure.

Nous avons écrit un analyseur en Python qui parcourt ces fichiers et produit deux structures. La première est le graphe : 193 nœuds, chacun avec son type, son annotation éventuelle et l’intégralité de ses paramètres sous forme de dictionnaire, et 219 connexions typées par port. La seconde est le schéma de chaque port de sortie, c’est-à-dire le nom, le type et le taux de valeurs manquantes de chaque colonne, que nous extrayons des fichiers de spécification que KNIME sérialise avec l’état d’exécution. Cette seconde structure nous a été décisive : elle fournit, pour chaque nœud, le lignage au sens de Cui et Widom (2001), sans avoir à ré-exécuter quoi que ce soit.

L’analyse établit d’abord la nature du pipeline. Vingt-et-un types de nœuds sont employés (table 1). Les vingt lectures se font par la paire DB Table Selector et DB Reader, sans requête personnalisée : chaque table est rapatriée entière, puis traitée en mémoire. Aucun nœud de calcul délégué à la base, comme DB GroupBy, DB Joiner ou DB Row Filter, n’est présent, alors que l’outil les propose. Le placement du calcul, dont la section 3.1 rappelait qu’il est traité par la littérature comme un problème d’optimisation, a donc été ici une décision implicite : tout se fait localement. Les douze écritures sont configurées en suppression-recréation de la table cible à chaque exécution, et la base propre est entièrement réécrite quatre fois par jour.

Type de nœud KNIME	Nb	Rôle dans le pipeline
Rule Engine	21	règles métier conditionnelles (familles, régions, indicateurs)
DB Table Selector / DB Reader	20 + 20	lecture d’une table entière de la base brute
String Manipulation	20	nettoyage de chaînes (espaces, concaténations)
String to Date&Time	20	conversion des dates stockées en texte yyyyMMdd (échec → manquant)
String to Number	15	conversion des montants et quantités stockés en texte (échec → manquant)
Joiner	14	jointures interne ou externe gauche (13 en comparaison stricte)
Row Filter	13	filtres de lignes
DB Writer	12	écriture d’une table de la base propre (suppression-recréation)
GroupBy	6	agrégations (Sum, Mean, Maximum, First, Last) et dédoublonnage
Math Formula · Column Filter	6 · 6	calculs arithmétiques · sélection de colonnes
Constant Value Column Appender	4	colonne constante (source d’origine)
Column Renamer · Date&Time Difference	3 · 3	renommages · écarts de dates en jours
Concatenate · Duplicate Row Filter · Missing Value · SQL Server Connector	2 · 2 · 2 · 2	union · dédoublonnage avec sélection · remplacement des manquants · connexions
Column Name Replacer · Column Resorter	1 · 1	renommage par motif · ordre des colonnes

Table 1 — Les vingt-et-un types de nœuds du workflow, par fréquence, et leur rôle.

L’analyse nous renseigne aussi sur la politique d’erreur, et corrige au passage une intuition que nous avions. Les vingt conversions de texte en date sont configurées pour produire une valeur manquante en cas d’échec, de même que les quinze conversions en nombre : une date mal formée ne fait donc pas échouer le nœud qui la lit. Les échecs complets constatés en exploitation, avec arrêt de la tâche, courriel et rapport figé sur les données de la veille, proviennent d’autres points. Les douze écritures en base sont en mode « échec sur erreur », et treize des quatorze jointures comparent les types de façon stricte. L’unité de défaillance reste l’exécution entière, mais elle est déclenchée plus loin dans le graphe que là où l’anomalie entre.

4.2. Douze branches, un tronc commun

Le graphe se décompose en douze branches, une par table produite (table 2). Elles sont très inégales : Opportunity mobilise 44 nœuds en amont de son écriture, dont 38 de traitement, quand ARTICLE en mobilise 5, dont un seul de traitement. Les branches ne sont pas indépendantes, puisque 23 nœuds servent à au moins deux branches et 21 à au moins trois. Ce tronc commun, qui lit les affaires, les commandes et les clients, les nettoie, les joint et recalcule le montant, alimente Affaire, Commande, BL et Prod. Dans KNIME, il est exécuté une fois et matérialisé ; toute traduction fidèle devra préserver cette propriété, faute de quoi le même calcul sera refait quatre fois.

Table produite	Amont	Traitement	Joint.	Agrég.	Tables brutes lues
Opportunity	44	38	0	0	OPPORTUNITY, HISTO_OPPORTUNITE
BL	33	23	4	2	AFFAIRE, BL, CLIENT, COMMANDE
BC_Fournisseur	31	21	3	2	BC_FOURNISSEUR, FOURNISSEUR, RECBL,

Table produite	Amont	Traitement	Joint.	Agrég.	Tables brutes lues
					STOCK
Commande	25	17	4	2	AFFAIRE, CLIENT, COMMANDE
Compta	24	16	2	1	AFFAIRE, BL, CLIENT
Prod	24	16	2	0	AFFAIRE, CLIENT, COMMANDE
Affaire	21	13	2	1	AFFAIRE, CLIENT, COMMANDE
OFFRE	18	12	0	0	OFFER, HISTO_DEVIS
ADESIGN_Projet	18	10	2	0	ADESIGN_PROJECTS, ADESIGN_CLIENTS, ADESIGN_GEOLOCATIONS
ADESIGN_Client	8	4	0	0	ADESIGN_CLIENTS
Gamme	6	2	0	0	GAMME
ARTICLE	5	1	0	0	ARTICLE

Table 2 — Les douze branches du workflow : nœuds en amont de l'écriture, dont nœuds de traitement, jointures et agrégations, et tables brutes lues.

Les six agrégations méritent une attention particulière, parce qu'elles sont l'endroit où la sémantique d'un nœud dépend de ce que le moteur promet. L'une d'elles n'agrège rien et sert de dédoublonnage. Les cinq autres emploient, à côté de sommes, de moyennes et de maximums, les méthodes « First » et « Last » de KNIME, c'est-à-dire la première ou la dernière valeur rencontrée dans le groupe, dans l'ordre où les lignes ont été lues. Or les lignes sont lues sans clause ORDER BY. Nous avons conservé ces méthodes telles quelles dans la représentation intermédiaire ; la section 6 montrera ce qu'il en a coûté à la chaîne d'origine.

4.3. La représentation intermédiaire

De ces deux structures nous dérivons la représentation pivot, au sens de Böhm et al. (2026) : pour chaque table produite, la liste ordonnée par tri topologique des opérateurs qui y conduisent, chacun décrit par son type générique, ses paramètres effectifs rendus lisibles et le schéma de sa sortie. Nous en donnons ci-dessous un extrait pour la branche la plus courte. Le numéro de nœud est conservé comme clé de traçabilité, mais rien dans la description ne présuppose KNIME : « regroupe par COBC ; agrégat : Last(DATEREC) » se lit et se traduit indépendamment de l'outil.

```
==================== PIPELINE → ARTICLE  (3 nœuds)
  #80   DB Table Selector    in=          «Article» Table dbo.ARTICLE
  #84   DB Reader            in=#80       Lecture de la table sélectionnée
  #152  String to Number     in=#84       Convertit PRIXART en Double (séparateur « . », échec →
missing)
  #105  DB Writer            Écrit dbo.ARTICLE (DROP + CREATE, REMOVE_TABLE)

==================== PIPELINE → BC_Fournisseur  (29 nœuds)  — extrait
  #178  String to Date_Time  in=#101      Convertit DATEREC en DATE format « yyyyMMdd » (erreur
→ SET_MISSING)
  #177  GroupBy              in=#178      Regroupe par COBC ; agrégats : Last(DATEREC)
  #79   Joiner               in=#177,#184 Jointure INNER sur COBC = COBC (comparaison STRING) ;
colonnes gauche : COBC, ART, QTE, … ; droite : DATEREC, …
  #96   Date_Time Difference in=#94       « Nb_jours_retard » = Date_reception − Livraison_prevu
en DAY
  #31   Rule Engine          in=#96       Ajoute « En_retard » : $Nb_jours_retard$ > 0 => 1 |
TRUE => 0
```

Cette représentation répond à notre première sous-question par l'affirmative, avec une réserve. Elle a suffi à engendrer les douze requêtes cibles : chaque étape M de la section 5 porte en commentaire le numéro de l'opérateur pivot qu'elle réalise, et le lignage colonne par colonne est complet, puisque toute colonne produite est rattachable à ses colonnes sources. Mais elle est descriptive et non exécutable : elle ne fixe pas la sémantique des opérateurs dont KNIME lui-même ne fixe pas la sémantique. « Last » y

figure comme « Last ». Notre traduction a donc dû interpréter en trois endroits, et c’est la validation, non la représentation, qui nous a dit si l’interprétation était juste.

5. Reconstruction dans Power Query

Nous avons mené la reconstruction requête par requête à partir de la représentation intermédiaire, avec deux contraintes fixées par l’entreprise : conserver à l’identique les noms de tables, de colonnes et de mesures, afin que les visuels de l’ancien rapport se copient dans le nouveau sans retouche ; et livrer un modèle complet, non un prototype.

5.1. Architecture du modèle migré

Le modèle migré, BI_V2, compte 28 tables et 105 mesures, contre 28 tables, 108 mesures et 11 colonnes calculées pour l’ancien. Nous avons reporté les onze colonnes calculées en DAX dans Power Query, où elles sont calculées à l’actualisation plutôt qu’au chargement du modèle, et quatre mesures ont été retirées par décision de périmètre. Nous organisons les requêtes en trois couches (figure 4).

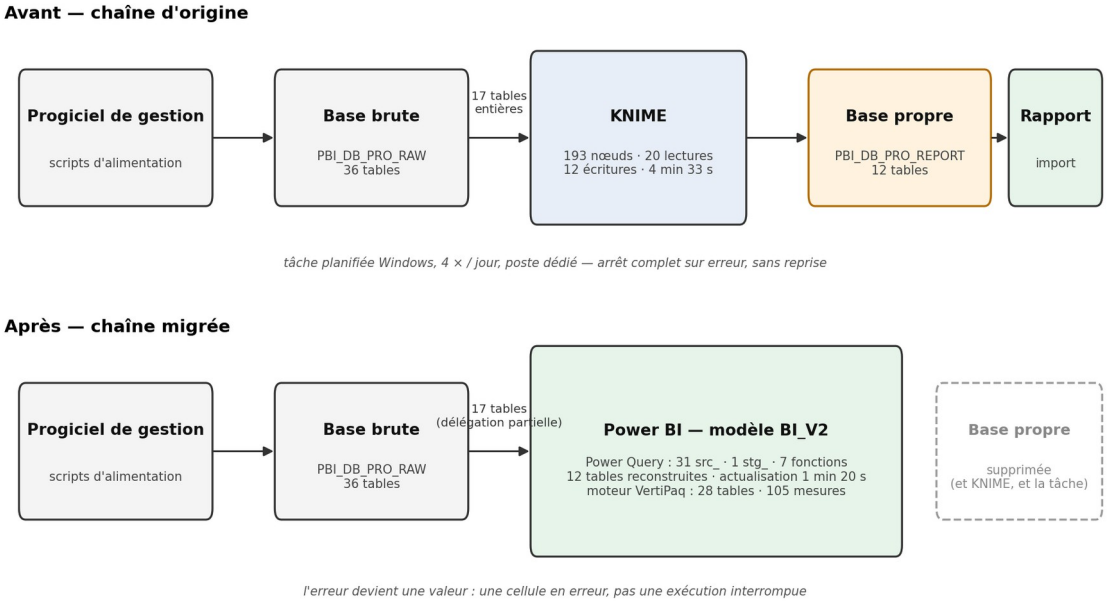


Figure 4 — Architecture avant et après. La base propre, le workflow et la tâche planifiée disparaissent ; les scripts d’alimentation de la base brute sont inchangés.

La couche source est faite de trente-et-une requêtes src_<TABLE>, une par table de la base brute, chacune réduite à la navigation Sql.Database(…){[Schema="dbo", Item=…]}[Data] et non chargée dans le modèle. Elles jouent le rôle des paires DB Table Selector et DB Reader, à une différence près : n’étant pas matérialisées, elles se composent avec les étapes qui les suivent, ce qui laisse au moteur la possibilité de déléguer. La couche intermédiaire comprend une requête de préparation partagée, stg_AffaireCommande, sept fonctions et une liste de constantes. La requête partagée réalise le tronc commun identifié en section 4.2 (nettoyage du code activité, jointure affaires-commandes, exclusion des commandes internes, conversions, recalcul du montant) et se termine par Table.Buffer, qui force son évaluation unique et sa conservation en mémoire. Sans cela, chacune des quatre tables qui s’appuient dessus la recalculerait : c’est l’équivalent explicite de la matérialisation qu’un flux de nœuds fait implicitement. Les fonctions factorisent les règles que le workflow répétait dans des nœuds Rule Engine : fnDateYYYYMMDD pour les dates stockées en texte, fnFamilleActivite, fnRegionRepresentant, fnTypeProduit et deux fonctions de correspondance des collaborateurs. La couche finale est faite des douze tables reconstruites, plus les tables de référence et de paramétrage que l’ancien modèle chargeait déjà par ailleurs.

5.2. Règles de traduction

La table 3 donne la correspondance que nous avons établie entre chaque type de nœud rencontré, sa classe dans la taxonomie de Vassiliadis et al. (2009) et la construction M qui le réalise. Nous l'avons écrite pour ce travail : la littérature n'en fournissait aucune pour ce couple de langages, et la documentation de l'éditeur décrit chaque fonction isolément, jamais comme cible d'un opérateur d'un autre outil.

Nœud KNIME	Classe	Construction M	Délég.
DB Table Selector + DB Reader	extraction	Sql.Database(…){[Schema, Item]}[Data] — requête src_ non chargée	oui
Column Filter · Row Filter	projection · sélection	Table.SelectColumns · Table.SelectRows	oui
String to Date&Time	tuple, 1:1	Table.TransformColumns + fnDateYYYYMMDD (try … otherwise null)	non
String to Number	tuple, 1:1	Table.TransformColumns + try Number.FromText(…) otherwise null	non
String Manipulation	tuple, 1:1	Table.TransformColumns + Text.Trim / Text.Replace / Text.Combine	partiel
Rule Engine	tuple, 1:1	Table.AddColumn + if … else, ou fonction nommée (fn*) si la règle est répétée	non
Math Formula · Date&Time Difference	tuple, 1:1	Table.AddColumn arithmétique · Duration.Days([a] − [b])	non
Constant Value Column Appender	tuple, 1:1	Table.AddColumn(t, nom, each "constante")	oui
Column Renamer · Column Name Replacer · Resorter	schéma	Table.RenameColumns · Table.TransformColumnNames · Table.SelectColumns (ordre)	oui
Missing Value	tuple, 1:1	Table.ReplaceValue(t, null, valeur, Replacer.ReplaceValue, colonnes)	non
Joiner	binaire, N×M	Table.NestedJoin(…, JoinKind.Inner | LeftOuter) + Table.ExpandTableColumn	non*
Concatenate	binaire, union	Table.Combine({a, b}) sur les colonnes communes	non*
GroupBy	agrégation, N:1	Table.Group(t, clés, {{col, each List.Sum | List.Average | List.Max | List.First}})	non*
Duplicate Row Filter	sélection, N:1	Table.Sort(critère) puis Table.Distinct(t, {clé}) — départage rendu explicite (6.4)	non*
DB Writer	chargement	— (la table est chargée dans le modèle ; Table.Buffer pour les résultats partagés)	—

** délégable en principe lorsque toutes les étapes amont le sont ; jamais atteint ici, la chaîne étant rompue par les conversions.*

Table 3 — Règles de traduction : type de nœud, classe dans la taxonomie de Vassiliadis et al. (2009), construction M retenue, et délégation possible.

Trois familles de nœuds se traduisent mécaniquement : les lectures, les transformations de tuple (conversion, manipulation de chaîne, formule, colonne constante, renommage) et les filtres. Le Rule Engine, qui porte l'essentiel de la logique métier, se traduit en chaîne conditionnelle ou, lorsque la même règle est répétée, en fonction nommée : ses vingt-et-une occurrences sont devenues sept fonctions et une dizaine d'expressions inline. Les jointures se traduisent en Table.NestedJoin suivi de Table.ExpandTableColumn, avec le même type de jointure et les mêmes colonnes retenues ; la comparaison stricte des types de KNIME n'a pas d'équivalent et n'en a pas besoin, les types étant fixés en amont.

Deux familles demandent une interprétation. Les agrégations « First » et « Last » n'ont pas de traduction fidèle, puisque leur résultat dépend d'un ordre que ni l'un ni l'autre moteur ne garantit. Nous les avons traduites par List.First, avec la même indétermination assumée, lorsque la valeur est constante

dans le groupe, et par List.Max lorsque l'intention métier lisible dans l'annotation du nœud était « la dernière date ». Le dédoublonnage avec sélection de ligne (Duplicate Row Filter) a été traduit par un tri explicite suivi de Table.Distinct sur la clé, en reprenant le critère de sélection configuré. Ces deux interprétations sont exactement les points que la validation devait éprouver.

5.3. La frontière de délégation

La section 3.1 nous laissait espérer que la réécriture dans un moteur à évaluation paresseuse déplacerait une partie du calcul vers SQL Server. L'analyse des requêtes produites nous conduit à une conclusion plus nuancée. La navigation vers la table et les sélections de colonnes et de lignes qui la suivent immédiatement se délèguent. Mais dans les douze requêtes, chaque table source subit, après au plus une sélection, une conversion de type (date à huit chiffres, nombre stocké en texte) écrite avec une fonction personnalisée ou un try … otherwise, et ni l'une ni l'autre ne se délègue, selon les règles que l'éditeur documente. La chaîne se rompt donc dès la deuxième ou troisième étape de chaque requête, et le volume rapatrié reste proche de celui de la chaîne d'origine. Notre analyse est ici statique ; l'éditeur de requêtes permet de la confirmer étape par étape, par l'option « Afficher la requête native », grisée dès que la délégation cesse.

La cause n'est pas dans la migration mais en amont : la base brute stocke les dates sous forme de texte à huit caractères, choix des scripts d'alimentation, et les montants sous forme de texte à séparateur décimal variable. Tant que ces conversions restent nécessaires, la délégation ne peut pas dépasser la première étape. Le gain de placement promis par la littérature est donc conditionné à la qualité du schéma source, ce que Yan et al. (2023) formulent en termes d'opérateurs partiellement délégables : dans notre cas, le premier opérateur ne l'est pas du tout. Nous en tirons une recommandation, typer les dates et les nombres dès l'alimentation, que nous plaçons en perspective puisqu'elle sort du périmètre de la migration.

5.4. Cas difficiles

Quatre points ont demandé plus qu'une traduction. Le premier est la sémantique d'erreur. Nous enveloppons les conversions dans try … otherwise null : une valeur illisible devient une cellule vide, la ligne survit et la table aussi. Le comportement est le même que celui des nœuds de conversion de KNIME, qui produisaient une valeur manquante ; la différence est en aval, où plus aucune écriture en base ne peut faire échouer l'exécution. Sur l'instantané de validation, nous avons examiné 436 837 cellules de date dans la base brute : une seule n'était pas convertible. Le mécanisme sert donc rarement, mais il détermine ce qui se passe le jour où il sert.

Le deuxième est la table de dates automatique. Power BI engendre, pour chaque colonne de date, une table calendaire cachée dont l'identifiant est un GUID propre au fichier, et une quarantaine de visuels de l'ancien rapport étaient liés à ces hiérarchies. Désactiver le mécanisme, comme le recommandent les guides de modélisation, aurait cassé ces visuels à la copie ; nous l'avons donc conservé, et le modèle migré compte trente-quatre tables de dates cachées et cinquante-et-une relations, dont dix-sept portées par le modèle lui-même.

Le troisième est le départage. Deux tables, OFFRE et Opportunity, fusionnent une source CRM et un historique Sellsy, puis dédoublonnent sur un numéro en gardant respectivement le montant le plus élevé et la création la plus récente. Lorsque deux lignes sont à égalité sur ce critère, rien ne dit laquelle garder. Notre première version a laissé ce cas au tri, comme KNIME le laissait à l'ordre de lecture ; la section 6 montre pourquoi c'était insuffisant et comment nous avons rendu la règle explicite.

Le quatrième est l'insensibilité à la casse du moteur de stockage. VertiPaq, le moteur en colonnes de Power BI, considère « ADIWATT », « Adiwatt » et « AdiWatt » comme une seule valeur et n'en conserve qu'une graphie. Ce n'est pas une propriété de Power Query, puisque la requête produit les trois graphies, mais du modèle qui les reçoit ; l'ancien modèle, qui importait la base propre, faisait déjà de même. Ce point a fixé le niveau d'équivalence que nous retenons en section 6.

6. Validation

Notre deuxième sous-question demandait à quel niveau d'équivalence valider, et par quel protocole. Nous décrivons ici le protocole, ses deux premiers passages, dont le premier a échoué pour une raison que notre étude bibliographique avait annoncée, et ce que le second a établi.

6.1. Protocole

Le test différentiel compare, table par table, les douze sorties de la chaîne d'origine, que nous appelons l'oracle, aux douze tables correspondantes du modèle migré, produites à partir du même état de la base brute. Trois captures sont nécessaires : la base brute, pour expliquer les écarts ; la base propre, qui est l'oracle ; et le contenu du modèle migré après actualisation, exporté depuis le moteur local de Power BI Desktop. Elles doivent tenir dans une même fenêtre, entre deux exécutions de la tâche planifiée, faute de quoi l'oracle ne correspond plus à son entrée.

Nous avons écrit pour ce travail un programme de comparaison qui rapproche les tables par nom et les colonnes par nom, et rend compte à trois niveaux déclarés avant l'expérience. N0 est l'égalité stricte des chaînes exportées ; il n'a d'autre usage que de montrer que les formats d'export diffèrent. N1 est l'égalité après normalisation : espaces de bordure supprimés, valeur manquante et chaîne vide confondues, dates ramenées à une forme canonique, nombres comparés avec une tolérance relative de 10^{-6}. N2 est N1 avec la casse des textes ignorée, pour la raison donnée en section 5.4, à savoir que c'est la sémantique du moteur de stockage et que c'était déjà celle de l'ancien modèle. Nous comparons en multi-ensembles, l'ordre des lignes ignoré. Lorsqu'une colonne ou une paire de colonnes est unique dans les deux tables, elle sert de clé naturelle : les lignes sont alors appariées et les écarts localisés colonne par colonne, avec des exemples. Nous avons éprouvé le programme sur une copie volontairement altérée d'une table, avec trois lignes supprimées, une valeur modifiée et un format de date changé, avant de l'appliquer.

Notre critère de réussite avait été fixé dans l'étude bibliographique : les douze tables coïncident avec l'oracle au niveau déclaré, et tout écart résiduel est expliqué, ou bien assumé comme correction de la chaîne d'origine et documenté comme tel. La suite de la section suit ce critère à la lettre.

6.2. Premier passage : l'instantané mixte

Notre premier passage a eu lieu le 15 septembre 2026 au matin. Les exports de la base brute ont couru de 9 h 47 min 55 s à 9 h 52 min 37 s, ceux de la base propre de 9 h 53 min 02 s à 9 h 53 min 13 s. Or l'historique de la tâche planifiée (figure 5) montre une exécution démarrée à 9 h 50 min 02 s et achevée à 9 h 54 min 35 s. Une partie de la base brute a donc été capturée avant le passage du workflow, une autre après, et un trou de trois minutes dans les horodatages de nos exports, entre deux tables, correspond au verrouillage des tables pendant l'alimentation. La base propre, elle, a été lue pendant que KNIME l'écrivait. Cet instantané ne permet aucune conclusion.

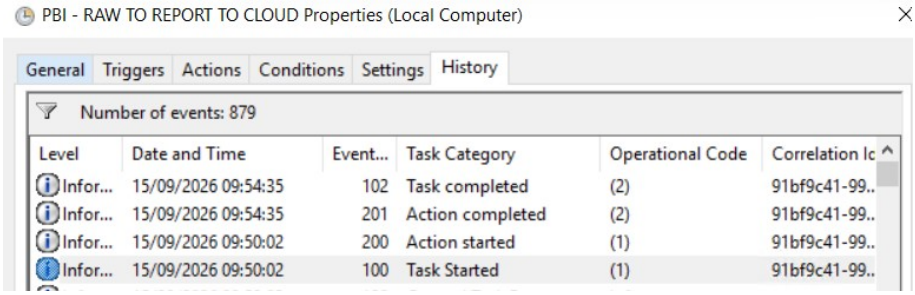


Figure 5 — Historique de la tâche planifiée : exécution du 15/09/2026 démarrée à 9 h 50 min 02 s, terminée à 9 h 54 min 35 s. Elle recouvre les premiers exports.

Nous rapportons cet épisode parce qu'il n'est pas une maladresse isolée mais la matérialisation d'une contrainte structurelle. L'oracle est un système vivant ; le figer suppose une fenêtre, et la fenêtre suppose de connaître l'ordonnancement. Nous avons complété le protocole d'une règle explicite, attendre dix

minutes après un « Task completed » et vérifier l'heure du prochain déclenchement, puis refait le passage.

6.3. Deuxième passage : résultats

Notre second passage a capturé la base brute de 10 h 14 min 19 s à 10 h 14 min 34 s, la base propre de 10 h 14 min 56 s à 10 h 15 min 05 s, et le modèle migré, actualisé sur cette base brute, de 10 h 40 min 42 s à 10 h 40 min 52 s. Aucune exécution de la tâche n'a eu lieu entre-temps, et la suite en fournit la preuve indirecte : neuf tables identiques à cent pour cent seraient impossibles si la base brute avait été réalimentée entre les captures. La table 4 donne le résultat au niveau N2.

Table	Lignes oracle	Lignes migré	Identiques	Écart	Verdict
Affaire	5 372	5 372	5 372	0	équivalent
Commande	5 372	5 372	5 372	0	équivalent
BL	5 052	5 052	5 052	0	équivalent
Compta	16 351	16 351	16 351	0	équivalent
Prod	14 672	14 672	14 672	0	équivalent
ARTICLE	4 780	4 780	4 780	0	équivalent
Gamme	81 456	81 456	81 456	0	équivalent (44 lignes : casse)
ADESIGN_Client	195	195	195	0	équivalent (25 lignes : casse)
ADESIGN_Projet	322	322	322	0	équivalent (104 lignes : casse)
BC_Fournisseur	38 854	38 854	38 474	380 (1,0 %)	défaut de l'origine — 6.4
Opportunity	24 935	24 935	21 962	2 973 (11,9 %)	défaut du migré, corrigé — 6.4
OFFRE	48 867	26 226	26 225	22 642	défaut de l'origine — 6.4

Table 4 — Test différentiel du 15/09/2026, deuxième passage, niveau N2. « Identiques » : lignes ayant un équivalent exact de l'autre côté ; l'écart est le nombre de lignes de l'oracle sans équivalent.

Neuf tables sur douze sont équivalentes ligne à ligne, dont les trois qui portent le plus de mesures, Affaire, Compta et Prod. Les trois autres divergent, et chacune pour une raison différente. Nous les avons analysées jusqu'au nœud responsable et prouvées sur les données ; elles font l'objet des trois paragraphes qui suivent, puis d'un arbitrage.

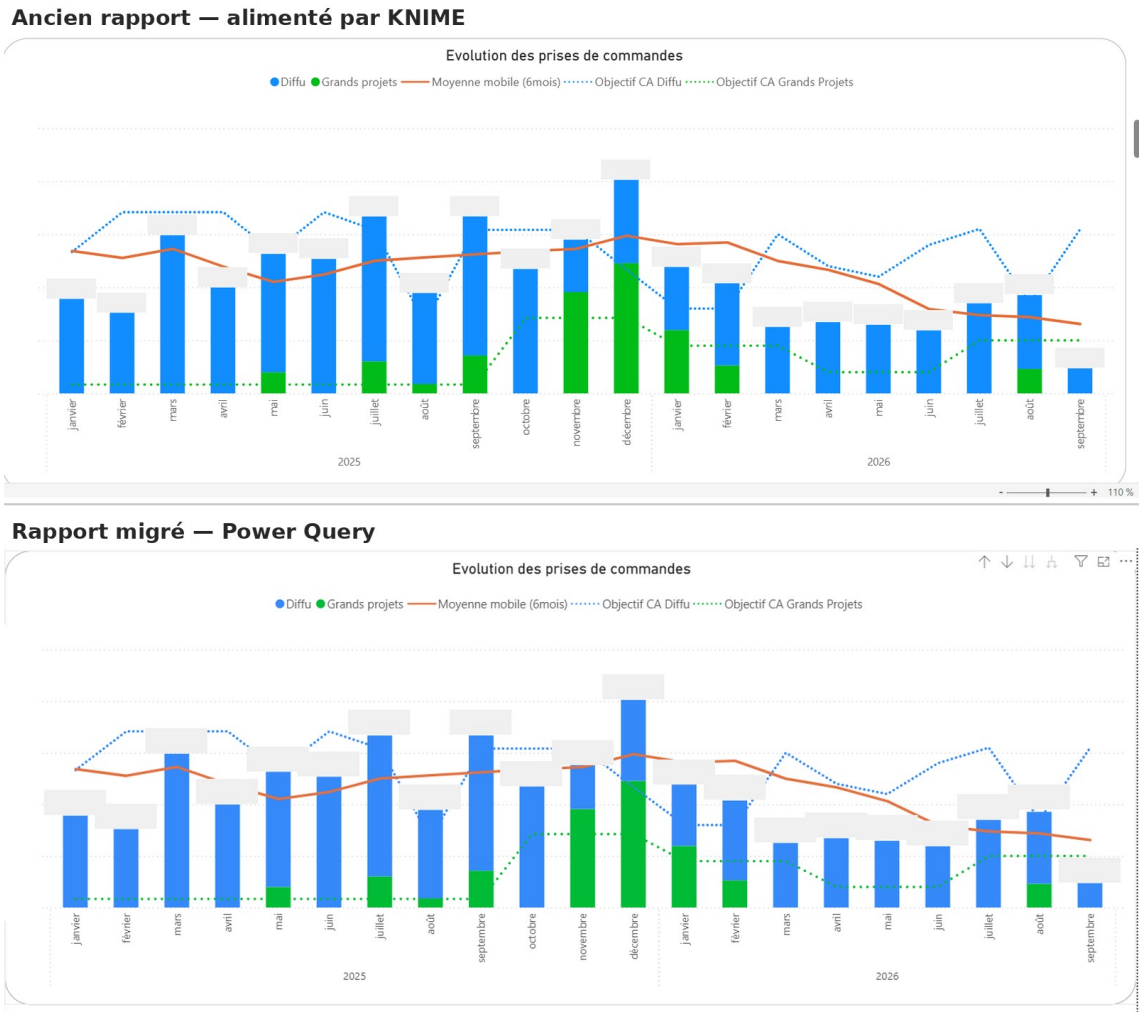


Figure 6 — Le même visuel, avec les mêmes filtres, dans l'ancien rapport et dans le rapport migré : l'évolution des prises de commandes, construite sur la table Affaire, équivalente à 100 %. Les montants sont masqués ; la forme des séries et la moyenne mobile sont identiques.

L'équivalence que nous établissons sur les tables se retrouve au niveau où l'utilisateur la voit. La figure 6 montre le même visuel dans les deux rapports, les visuels ayant été copiés sans retouche grâce à la conservation des noms de tables et de mesures. Pour les neuf tables équivalentes, cette identité est la règle. Pour les trois autres, la divergence est également visible à l'écran, comme nous le montrons pour Opportunity dans la section suivante.

6.4. Trois divergences, trois causes

BC_Fournisseur : « Last » n'est pas « max ». 380 lignes diffèrent, toutes sur la date de réception et les colonnes qui en dérivent, c'est-à-dire le nombre de jours de retard et l'indicateur de retard. Le nœud GroupBy #177 agrège la date de réception par commande fournisseur avec la méthode « Last » ; notre traduction, lisant dans l'annotation du nœud l'intention « dernière date de réception », a pris List.Max. Les deux ne coïncident que si les réceptions sont lues dans l'ordre chronologique, ce que rien ne garantit. La preuve est directe : pour les 12 075 commandes fournisseur de l'instantané, la date retenue par l'oracle est égale à la dernière ligne lue dans 12 075 cas, et à la date maximale dans 11 994 cas seulement. Pour 81 commandes, soit 378 lignes, la chaîne d'origine retient une réception antérieure à la dernière et calcule un retard faux. Ainsi, pour la commande 10234, les réceptions sont lues dans l'ordre 22/12/2025, 22/12/2025, 13/01/2026 six fois, puis 22/12/2025 : KNIME retient le 22/12/2025, Power Query le 13/01/2026. Nous sommes exactement dans le cas décrit par Shi et al. (2016), une implémentation qui suppose déterministe ce que la spécification ne promet pas. L'oracle est ici faux, et il l'est de façon instable, puisqu'une réorganisation physique de la table des réceptions aurait changé les chiffres du

rapport sans qu'aucune donnée ne change. Nous retenons donc un défaut de la chaîne d'origine, assumé comme correction.

Opportunity : l'égalité parfaite n'a pas de vainqueur. 2 973 lignes diffèrent. Les deux implémentations dédoublonnent sur le numéro d'opportunité en gardant la création la plus récente, le nœud #250 par sélection du maximum de CREATEDATE et notre requête M par tri décroissant puis Table.Distinct. Mais 2 910 opportunités existent dans les deux sources, CRM et historique Sellsy, avec exactement la même date de création : aucune des deux implémentations ne dit alors quoi faire. KNIME garde la première ligne rencontrée, qui est la ligne CRM parce que cette branche entre par le premier port du nœud de concaténation ; notre tri M, qui n'est pas stable sur les égalités, laisse passer la ligne Sellsy. Le modèle migré perd alors, pour ces lignes, les colonnes que seul le CRM renseigne, à savoir la date de signature, l'assistante, le secteur public, le nombre de modules, l'altitude et le statut, et les remplace par les constantes de la branche Sellsy. Une soixantaine de lignes supplémentaires relèvent du même mécanisme à l'intérieur d'une même source. KNIME n'était pas plus déterministe que nous sur ce point ; son résultat était simplement stable par effet de bord. Nous retenons donc un défaut de notre modèle migré, à corriger par une règle de départage explicite.

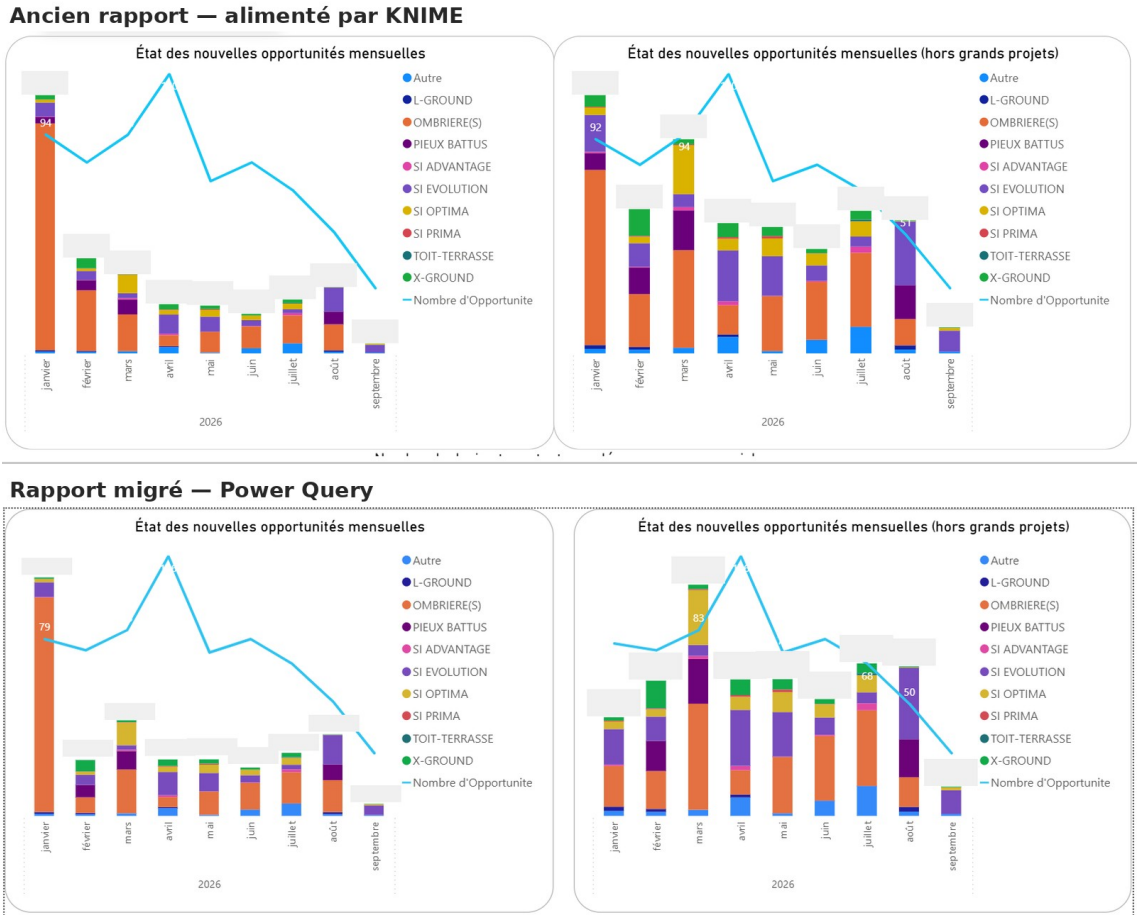


Figure 7 — La divergence Opportunity vue de l'utilisateur, avant correctif : l'état des nouvelles opportunités mensuelles dans l'ancien rapport (haut) et dans le rapport migré (bas). Montants masqués ; les nombres d'opportunités affichés diffèrent, 94 contre 79 en janvier, parce que les 2 910 lignes qui ont basculé de source portent un autre type de produit et un autre statut.

La figure 7 montre ce que cette divergence fait au rapport : le nombre d'opportunités de janvier passe de 94 à 79, et la répartition par type de produit change, parce que les lignes qui ont basculé vers la source Sellsy y portent un autre libellé d'axe. C'est la seule des douze tables où l'utilisateur aurait vu la migration, et c'est aussi la seule divergence imputable à notre modèle migré. Nous la corrigeons par la règle donnée plus bas.

OFFRE : la configuration affichée n'est pas le comportement. La chaîne d'origine produit 48 867 lignes, notre modèle migré 26 226. Le nœud Duplicate Row Filter #265 est configuré, dans son fichier de

paramètres, pour dédoublonner sur le seul numéro d'offre en gardant le montant le plus élevé, et c'est ce que notre requête M reproduit. Mais l'oracle contient 22 626 numéros présents deux fois, une ligne CRM et une ligne Sellsy. Sur les données, la clé effective du nœud est le couple numéro-libellé : il ne reste aucun doublon sur ce couple, et le libellé n'existe que côté CRM. L'explication tient à la politique de filtrage de colonnes de KNIME. Le nœud avait été configuré par exclusion, c'est-à-dire que toute colonne non explicitement exclue est incluse dans la clé, et la colonne de libellé, ajoutée à la branche CRM après cette configuration, a rejoint la clé sans que rien ne le signale à l'écran. Chaque offre présente dans les deux systèmes est donc comptée deux fois depuis lors. Nous retenons un défaut de configuration de la chaîne d'origine. Ce cas est le plus instructif des trois, parce qu'aucune lecture du workflow, ni à l'écran ni dans le fichier, ne permettait de le voir : seule la comparaison des sorties l'a révélé.

Nous avons soumis ces trois divergences à arbitrage. Pour BC_Fournisseur et OFFRE, le comportement du modèle migré, c'est-à-dire la date maximale et une ligne par offre au montant le plus élevé, est retenu comme le comportement voulu, et l'écart est documenté comme correction de la chaîne d'origine ; les indicateurs fondés sur les offres baissent en conséquence, ce qui devra être annoncé aux utilisateurs. Pour Opportunity, nous avons rendu la règle de départage explicite : à date de création égale, priorité à la ligne CRM, puis au montant le plus élevé. Elle tient en trois étapes M, que nous reproduisons ci-dessous, et a été appliquée au modèle.

```
// #250 doublons sur NUMBER : création la plus récente ; à date égale, priorité au CRM,
// puis montant le plus élevé (règle de départage explicite — arbitrage du 15/09/2026)
Priorite = Table.AddColumn(Union, "PrioSource", each if [SOURCE] = "OPP" then 0 else 1,
Int64.Type),
Trie     = Table.Buffer(Table.Sort(Priorite,
            {{"CREATEDATE", Order.Descending}, {"PrioSource", Order.Ascending}, {"AMOUNT",
Order.Descending}})),
Unique   = Table.RemoveColumns(Table.Distinct(Trie, {"NUMBER"}), {"PrioSource"}),
```

[Troisième passage — résultat à insérer après confirmation sur un nouvel instantané. Attendu : onze tables équivalentes, OFFRE expliquée, et pour Opportunity un résidu limité aux doublons internes départagés par le montant là où KNIME départageait par l'ordre de lecture.]

6.5. Ce qui n'est pas un écart

Trois catégories de différences apparaissent au niveau N1 et disparaissent au niveau N2 ou dans la tolérance. Nous les rapportons pour que le lecteur puisse juger du niveau que nous avons retenu. La casse des textes, d'abord : 44 lignes de Gamme et 129 lignes des tables ADESIGN ne diffèrent que par la graphie d'un libellé, « ADIWATT » contre « Adiwatt », parce que le moteur de stockage ramène toutes les graphies d'une valeur à la première rencontrée. La virgule flottante, ensuite : une ligne d'Affaire et une de BL diffèrent au neuvième chiffre significatif d'un prix moyen, 11 877,085454546 contre 11 877,085454545. Une colonne, enfin : UPDATEDATE, que le workflow écrivait dans Opportunity et qu'aucune version du rapport n'a jamais lue.

Nous soulignons que le niveau d'équivalence retenu n'est pas le plus fort possible. La réplication au bit près, au sens de Pritchard et Wicenec (2024), est hors d'atteinte entre deux moteurs qui ne représentent ni les nombres ni les chaînes de la même façon. L'équivalence utile est celle des valeurs que l'utilisateur du rapport peut voir, et c'est ce niveau que N2 formalise. Le choix est métier avant d'être technique.

7. Mesures et bilan

Notre troisième sous-question demandait ce que l'on gagne et ce que l'on perd, mesurément. Nous donnons les mesures disponibles à la date de rédaction avec leur nombre d'observations ; certaines sont uniques et devront être répétées.

7.1. Durée, volume, artefacts

Grandeur	Chaîne d'origine	Chaîne migrée	Source
Durée du traitement	4 min 33 s (workflow seul)	1 min 20 s (actualisation complète)	historique de la tâche ; Desktop
Lignes lues (17 tables brutes)	343 645	343 645	instantané du 15/09
Lignes produites (12 tables)	246 217	223 576 (OFFRE dédoublonnée)	instantané du 15/09
Étages entre la base brute et le rapport	2 (workflow, base propre)	0	architecture
Artefacts de logique métier	193 nœuds, format XML propre à l'outil	52 requêtes M, texte	fichiers
Colonnes calculées en DAX	11	0	modèles TMDL
Unité de défaillance sur donnée invalide	l'exécution	la cellule	construction, 7.2
Fréquence d'actualisation	4 / jour	4 / jour (inchangée)	planification

Table 5 — Mesures comparées. Les durées reposent sur une observation chacune (n = 1).

La durée du traitement passe de 4 min 33 s, qui est l'exécution du workflow seule relevée dans l'historique de la tâche, hors alimentation de la base brute, à 1 min 20 s pour l'actualisation complète du modèle migré, laquelle comprend le chargement dans le moteur de stockage. La comparaison est conservatrice, puisque l'ancienne chaîne exigeait après le workflow une actualisation du rapport depuis la base propre que nous ne comptons pas. Le gain est réel mais, comme notre étude bibliographique l'anticipait, il ne porte pas d'abord sur la vitesse : la chaîne d'origine était déjà rapide.

Le volume traité est inchangé : 343 645 lignes lues dans dix-sept tables brutes, 246 217 lignes produites dans douze tables. La section 5.3 a expliqué pourquoi la délégation à la source n'a pas réduit le volume rapatrié, et pourquoi la cause est en amont de la migration.

Les artefacts, eux, changent de nature. Disparaissent une base de douze tables réécrite quatre fois par jour, un workflow de 193 nœuds dans un format propre à l'outil, une tâche planifiée, et un paradigme de transformation sur deux. Apparaissent cinquante-deux requêtes M pour la partie migrée, dans un fichier unique versionnable en texte, dont trente-et-une sont de simples navigations vers les tables sources. Les onze colonnes calculées en DAX ont rejoint Power Query, et le modèle n'en compte plus aucune. Le poste dédié reste nécessaire aux scripts d'alimentation de la base brute, tant que ceux-ci ne sont pas déplacés à leur tour.

7.2. Comportement en présence d'une donnée invalide

Nous n'avons injecté aucune anomalie en production, et ce point n'a donc pas fait l'objet d'une expérience contrôlée. Le comportement se déduit de la construction : dans le modèle migré, toute conversion est enveloppée dans try … otherwise et aucune étape n'écrit en base, de sorte qu'il n'existe plus d'opérateur capable d'interrompre l'actualisation sur une valeur de donnée. Dans la chaîne d'origine, les conversions produisaient elles aussi des valeurs manquantes, mais douze écritures en mode « échec sur erreur » et treize jointures à comparaison stricte de types restaient des points d'arrêt possibles. Sur l'instantané, une seule cellule de date sur 436 837 n'était pas convertible, et elle est devenue une cellule vide des deux côtés.

7.3. Ce que la migration a révélé

Le résultat que nous n'attendions pas ne porte pas sur ce que la migration a reproduit, mais sur ce qu'elle a mis au jour. En cherchant à établir l'équivalence, le test différentiel nous a fait trouver deux défauts dans la chaîne que nous prenions pour référence : 81 commandes fournisseur avec un retard calculé sur une mauvaise date, et 22 626 offres comptées deux fois. Aucun des deux n'était visible dans le workflow, le premier parce que la méthode « Last » a l'air de faire ce qu'on lui demande, le second parce que la configuration affichée du nœud ne dit pas sa clé effective. Aucun des deux n'avait été

signalé par les utilisateurs. Le test différentiel a donc servi ici à deux choses : valider la migration, et auditer l'original. C'est un usage que la littérature de la section 3.4 ne décrit pas, parce qu'elle suppose l'oracle correct.

7.4. Limites

Nous voyons quatre limites à ce travail. Les durées reposent sur une observation chacune et devront être répétées. Le troisième passage de validation, après correctif, reste à réaliser. La délégation à la source, argument central du premier axe de notre étude bibliographique, n'a produit ici aucun gain mesurable, pour une raison identifiée mais extérieure à la migration. Enfin, les assertions déclaratives de Schelter et al. (2018), que nous avions prévues comme surveillance continue après bascule, ne sont pas encore en place : notre validation est celle d'un instantané, non d'une exploitation.

8. Conclusion et perspectives

Nous avons reconstruit, dans le moteur de transformation de l'outil de restitution, un pipeline décisionnel de 193 nœuds bâti dans un outil de flux visuel, et nous l'avons validé par test différentiel contre les sorties de la chaîne d'origine. Les trois sous-questions posées par notre étude bibliographique reçoivent une réponse.

Sur la représentation intermédiaire, le format de sérialisation du workflow, analysé en graphe et en schémas de ports, fournit une représentation descriptive suffisante pour engendrer les douze requêtes cibles avec un lignage complet. En revanche, elle ne fixe pas la sémantique des opérateurs que l'outil d'origine ne fixait pas lui-même, et notre traduction a dû interpréter en trois endroits. Sur le niveau d'équivalence et le protocole, le test différentiel en multi-ensembles, à tolérance numérique déclarée et casse ignorée, sur un instantané pris entre deux exécutions, s'est révélé praticable à l'échelle d'une chaîne complète : il a établi neuf équivalences sur douze, puis expliqué les trois écarts jusqu'au nœud responsable. Sur le gain, la durée est divisée par trois, le volume est inchangé, l'unité de défaillance passe de l'exécution à la cellule, et la logique métier existe désormais sous une forme textuelle et versionnable.

Nous retenons deux enseignements qui dépassent le cas traité. Le premier est que l'oracle d'une migration n'est une référence que pour l'équivalence, non pour la correction : deux de nos trois divergences étaient des défauts de l'original, dont l'un ne pouvait être vu que dans les sorties. Le test différentiel, présenté par la littérature comme un instrument de validation, est aussi un instrument d'audit. Le second est que la sémantique d'un pipeline visuel n'est pas dans ce qu'il affiche : une méthode d'agrégation dépendante de l'ordre de lecture et une politique de filtrage de colonnes par exclusion ont suffi à produire, silencieusement, des chiffres faux pendant des années.

Trois perspectives s'ouvrent. La première est technique : typer les dates et les montants dès l'alimentation de la base brute déplacerait la frontière de délégation de la deuxième étape vers les jointures et les agrégations, et donnerait enfin sa portée au premier axe de notre étude. La deuxième est opérationnelle : mettre en place, à chaque actualisation, un jeu d'assertions déclaratives sur le nombre de lignes, l'unicité des clés et les bornes de dates, pour que la validation ne soit plus celle d'un instantané mais celle d'une exploitation. La troisième est méthodologique : le protocole et le programme de comparaison que nous avons écrits ne dépendent ni de KNIME ni de Power Query, et s'appliquent à toute migration disposant d'une chaîne d'origine encore exécutable. C'est peut-être leur meilleur usage, avant même de migrer, pour savoir ce que l'on reproduit.

Références

Les quinze références de l'étude bibliographique, toutes librement accessibles en ligne. Les documentations d'éditeurs et les outils logiciels sont listés en annexe.

Armbrust, M. et al. (2015). *Spark SQL: Relational Data Processing in Spark*. SIGMOD '15. https://people.csail.mit.edu/matei/papers/2015/sigmod_spark_sql.pdf

Böhm, T. et al. (2026). *Model-Driven Legacy System Modernization at Scale*. ReCode '26 @ ICSE · arXiv:2602.04341. https://arxiv.org/abs/2602.04341

Cui, Y., Widom, J. (2001). *Lineage Tracing for General Data Warehouse Transformations*. VLDB '01. https://www.vldb.org/conf/2001/P471.pdf

Haas, L. M., Kossmann, D., Wimmers, E. L., Yang, J. (1997). *Optimizing Queries across Diverse Data Sources*. VLDB '97, 276–285. https://www.vldb.org/conf/1997/P276.PDF

He, Y., Zhao, P., Wang, X., Wang, Y. (2024). *VeriEQL: Bounded Equivalence Verification for Complex SQL Queries with Integrity Constraints*. OOPSLA 2024 · arXiv:2403.03193. https://arxiv.org/abs/2403.03193

Johnston, W. M., Hanna, J. R. P., Millar, R. J. (2004). *Advances in Dataflow Programming Languages*. ACM Computing Surveys 36(1), 1–34. https://www.cs.tufts.edu/comp/150CMP/papers/dataflow-johnston-acm04.pdf

Pritchard, N. J., Wicenec, A. (2024). *Formal Definition and Implementation of Reproducibility Tenets for Computational Workflows*. Préprint arXiv:2406.01146. https://arxiv.org/abs/2406.01146

Rigger, M., Su, Z. (2020). *Testing Database Engines via Pivoted Query Synthesis*. USENIX OSDI 2020 · arXiv:2001.04174. https://arxiv.org/abs/2001.04174

Schelter, S., Lange, D., Schmidt, P., Celikel, M., Biessmann, F., Grafberger, A. (2018). *Automating Large-Scale Data Quality Verification*. PVLDB 11(12), 1781–1794. http://www.vldb.org/pvldb/vol11/p1781-schelter.pdf

Sculley, D. et al. (2015). *Hidden Technical Debt in Machine Learning Systems*. NeurIPS 2015. https://proceedings.neurips.cc/paper_files/paper/2015/file/86df7dcfd896fcaf2674f757a2463eba-Paper.pdf

Shi, A., Gyori, A., Legunsen, O., Marinov, D. (2016). *Detecting Assumptions on Deterministic Implementations of Non-deterministic Specifications*. IEEE ICST 2016 (outil NonDex). https://www.cs.cornell.edu/~legunsen/pubs/ShiETAL16NonDex.pdf

Simitsis, A., Vassiliadis, P., Sellis, T. (2005). *Optimizing ETL Processes in Data Warehouses*. ICDE 2005, 564–575. https://www.cs.ubc.ca/~rap/teaching/534P/2011/readings/etl.pdf

Vassiliadis, P., Simitsis, A., Baikousi, E. (2009). *A Taxonomy of ETL Activities*. DOLAP '09. https://www.cs.uoi.gr/~pvassil/publications/2009_DOLAP_ETL/DOLAP_2009_ETL.pdf

Yan, C., Lin, Y., He, Y. (2023). *Predicate Pushdown for Data Science Pipelines*. PACMMOD 1(2) · SIGMOD 2023. https://www.microsoft.com/en-us/research/wp-content/uploads/2023/05/predicate_pushdown_final.pdf

Zhou, Q., Arulraj, J., Navathe, S., Harris, W., Wu, J. (2022). *SPES: A Symbolic Approach to Proving Query Equivalence Under Bag Semantics*. ICDE 2022 · arXiv:2004.00481. https://arxiv.org/abs/2004.00481

Annexe A — Outils écrits pour ce travail

figer_oracle.ps1. Exporte, dans un dossier daté, les 17 tables brutes lues par le workflow et les 12 tables propres qu'il produit (CSV UTF-8, manifeste avec nombre de lignes et empreinte SHA-256). Aucun identifiant n'y est stocké.

comparer.py. Test différentiel : rapprochement des tables et des colonnes par nom, niveaux N0 / N1 / N2, sémantique des multi-ensembles, détection automatique d'une clé naturelle, écarts localisés par colonne avec exemples, rapport Markdown et JSON. Accepte les exports SQL, DAX Studio et « Copier la table » de Power BI.

parse.py / summarize.py. Analyse du format de sérialisation KNIME : graphe des 193 nœuds avec leurs paramètres, schémas des ports de sortie, tri topologique par table produite, représentation intermédiaire lisible (section 4.3).

exporter_pbi.ps1. Export des tables du modèle depuis le moteur local de Power BI Desktop via ADOMD ou OLE DB. Inutilisable sur le poste de l'étude (version Store sans bibliothèque cliente) ; remplacé par DAX Studio, outil libre.

Annexe B — Documentations d'éditeurs et outils logiciels

Sources techniques mobilisées ; non évaluées par les pairs, elles sont tenues à l'écart des références.

Microsoft Learn — Query folding (Power Query). Définition par l'éditeur du mécanisme de délégation, des étapes délégables et du point de rupture. Sections 2.3 et 3.2. https://learn.microsoft.com/fr-fr/power-query/query-folding-basics

Microsoft Learn — Spécification du langage M. Description de référence du modèle d'exécution du langage de transformation cible : fonctionnel, à valeurs immuables, à évaluation paresseuse. Section 2.2. https://learn.microsoft.com/fr-fr/powerquery-m/power-query-m-language-specification

KNIME — Database Extension Guide. Documentation des nœuds qui composent du SQL sans matérialiser. Elle établit que l'outil de flux sait déléguer, et que le pipeline étudié n'en fait pas usage. Sections 2.2 et 3.2. https://docs.knime.com/ap/latest/databases/

Deequ (AWS Labs). Implémentation de référence de Schelter et al. (2018) : forme concrète d'une contrainte de qualité déclarative compilée en agrégations. Sections 3.5 et 4.3. https://github.com/awslabs/deequ

Great Expectations. Assertions déclaratives sur les données, documentation de validation engendrée. Candidat pour les invariants métier, section 4.3. https://docs.greatexpectations.io/

data-diff (Datafold, projet archivé). Comparaison ligne à ligne entre bases hétérogènes ; référence outillée de la réconciliation, projet non maintenu. Section 4.3. https://github.com/datafold/data-diff

Cosette (démonstrateur). Prouveur d'équivalence de requêtes SQL utilisable en ligne ; illustre concrètement ce que « prouver » signifie dans la hiérarchie de la section 3.5. https://cosette.cs.washington.edu

DAX Studio. Outil libre de requêtage des modèles Analysis Services et Power BI ; utilisé pour exporter les douze tables du modèle migré (section 6.1). https://daxstudio.org/